

Методичні вказівки до практичного заняття

Тема: Парадигми ООП, проблема «комбінаторного вибуху» та патерни проектування.

1. Контекст та постановка задачі

Уявіть, що ви розробляєте рушій для 2D-графічного редактора (на кшталт Figma чи Canva), де потрібно реалізувати систему відображення базових геометричних фігур. У першій версії технічного завдання (ТЗ) є лише три форми: **Трикутник**, **Коло**, **Квадрат**. Для них створюється базовий абстрактний клас та три похідні:

```
class Shape {
public:
    virtual void display() = 0;
    virtual ~Shape() = default;
};

class Triangle : public Shape { /* ... */ };
class Circle : public Shape { /* ... */ };
class Square : public Shape { /* ... */ };
```

Згодом вимоги змінюються, і фігурам потрібно додати нові незалежні характеристики:

- **Розмір (Size):** Малий, Середній, Великий (3 варіанти).
- **Заливка (Fill):** Порожня, Напівзафарбована, Повністю зафарбована (3 варіанти).

2. Спроба №1: Наївне наслідування

Найбільш очевидний шлях, який спадає на думку — створити окремий клас під кожну можливу комбінацію властивостей: SmallEmptyTriangle, LargeFullCircle, MediumHalfFullSquare тощо.

У чому проблема? Загальна кількість комбінацій складе $3 \times 3 \times 3 = 27$ класів.

Комбінаторний вибух: Якщо згодом додасться ще одна характеристика, наприклад, 4 кольори, кількість класів зросте до $27 \times 4 = 108$. Таку систему неможливо підтримувати, код відображення постійно дублюється, а архітектура перетворюється на некероване "пекло наслідування".

3. Спроба №2: Додавання полів (Data-driven approach)

Наступна ідея — відмовитися від створення нових класів і зберегти стан фігури через звичайні змінні-прапорці всередині базового класу Shape:

```
class Shape {
protected:
    std::string size; // "small", "medium", "large"
    std::string fill; // "empty", "half-full", "full"
public:
    virtual void display() = 0;
};
```

Тоді всередині кожної конкретної фігури метод display() буде містити розгалужену логіку з умовними операторами:

```
void Triangle::display() {
    std::cout << "I am a ";
    if (size == "small") std::cout << "Small ";
    else if (size == "medium") std::cout << "Medium ";
    else if (size == "large") std::cout << "Large ";

    if (fill == "empty") std::cout << "Empty ";
    else if (fill == "half-full") std::cout << "Half-full ";
    else if (fill == "full") std::cout << "Full ";

    std::cout << "Triangle" << std::endl;
}
```

Чому такий підхід не працює у складних проектах?

1. **Порушення SRP (Принцип єдиної відповідальності):** Клас фігури (наприклад,

Triangle) тепер змушений знати все про логіку зафарбовування, масштабування чи обробку текстур, замість того, щоб відповідати лише за власну геометрію.

2. **Порушення ОСР (Принцип відкритості/закритості):** Якщо завтра з'явиться новий тип заливки (наприклад, градієнт), доведеться модифікувати код **усіх існуючих фігур**, додаючи туди нові блоки if/else.
3. **Дублювання логіки:** Алгоритми обробки розмірів чи типів заливки будуть ідентичними для кола, квадрата й трикутника, але вам доведеться копіювати ці перевірки у кожен клас.

4. Шлях ООП: Заміна наслідування композицією

Щоб уникнути жорсткої прив'язки, використовується фундаментальне правило архітектури: **«Ізолюйте те, що змінюється»**. Замість наслідування (Is-A) перевага віддається **композиції об'єктів (Has-A)**.

Властивості фігур виносяться в *окремі незалежні ієрархії класів*, а сам об'єкт фігури просто зберігає покажчики на них. Це дозволяє легко додавати нові характеристики, не чіпаючи код фігур, і навіть змінювати параметри об'єктів динамічно прямо під час виконання програми (Runtime).

5. Практичне завдання

Базовий інтерфейс:

```
#include <iostream>
#include <string>

class Shape {
public:
    virtual void display() = 0;
    virtual ~Shape() = default;
};
```

Завдання:

Необхідно розробити архітектуру, яка дозволяє уникнути комбінаторного вибуху та розгалужених if/else перевірок у фігурах. **Оберіть 2 з 3** запропонованих патернів

проектування та **повністю реалізуйте їх у коді на C++**. У результаті у вас має бути дві окремі незалежні програми, які вирішують цю задачу різними способами.

Патерни на вибір:

Патерн	Суть підходу	Вектор для реалізації
Bridge (Міст)	Поділ абстракції та її реалізації.	Винесіть характеристики в окремі інтерфейси Size та Fill. Об'єкт Shape повинен агрегувати покажчики на ці інтерфейси та делегувати їм виконання роботи. Кількість класів скоротиться до $3 + 3 + 3 = 9$. Докладніше: Bridge на Refactoring.Guru .
Strategy (Стратегія)	Ізоляція характеристик у вигляді змінних алгоритмів відображення.	Передавайте конкретні поведінки (наприклад, SizeStrategy або FillStrategy) як параметри безпосередньо в конструктор фігури або в метод display(). Докладніше: Strategy на Refactoring.Guru .
Decorator (Декоратор)	Динамічне розширення функціоналу об'єкта через шари обгортки.	Класи фігур залишаються повністю "чистими". Створюється базовий ShapeDecorator, від якого наслідуються конкретні модифікатори (SmallDecorator, FullDecorator). Конструювання

Патерн	Суть підходу	Вектор для реалізації
		об'єкта в <code>main()</code> виконується вкладеним принципом: <code>new FullDecorator(new SmallDecorator(new Triangle()));</code>. Докладніше: Decorator на Refactoring.Guru.

Вимоги до результату:

- При виклику методу `display()` для налаштованої фігури, консоль повинна виводити англomовний рядок: **"I am a Large Half-full Triangle"** (або аналогічний текст відповідно до комбінації).
- У функції `main()` потрібно продемонструвати працездатність обох обраних патернів щонайменше для 3-х різних комбінацій.

Обговорення рішень (Peer Review):

Після написання коду порівняйте свої рішення з роботами колег по групі (бажано з тими, хто обрав іншу пару патернів), та проаналізуйте наступні питання:

1. Який із реалізованих вами патернів створює менше накладних витрат по пам'яті під час конструювання об'єктів у купі?
2. Якщо замовник попросить додати нову фігуру `Hexagon` (Шестикутник), скільки класів доведеться дописати або змінити в обох ваших варіантах?
3. Якщо потрібно додати четверту характеристику — "Колір" (4 варіанти), скільки нових класів доведеться створити, і яка архітектура виявиться найбільш стійкою до таких змін?